

Web Application Development

❖ JavaScript

Ammar Ahmad Khan

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

In JavaScript we have the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed

The if Statement

Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Note that **if** is in lowercase letters. Uppercase letters (If or IF) will generate a JavaScript error.

Example

Make a "Good day" greeting if the hour is less than 18:00:

```
if (hour < 18) {  
    greeting = "Good day";  
}
```

The result of greeting will be:

Good day

The if Example

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript if</h2>
<p>Display "Good day!" if the hour is less than 18:00:</p>
<p id="demo">Good Evening!</p>
<script>
if (new Date().getHours() < 18) {
    document.getElementById("demo").innerHTML = "Good day!";
}
</script>
</body>
</html>
```

The else Statement

Use the else statement to specify a block of code to be executed if the condition is false.

Syntax:

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

Example

If the hour is less than 18, create a "Good day" greeting, otherwise "Good evening":

```
if (hour < 18) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

The result of greeting will be:

Good day

The else Example

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript if...else Example</h2>
<p>Check if a person is an adult or a minor:</p>
<p id="demo"></p>
<script>
let age = 16; // Change this value to test different cases
let message;
if (age >= 18) {
    message = "You are an adult.";
} else {
    message = "You are a minor.";
}
document.getElementById("demo").innerHTML = message;
</script>
</body>
</html>
```

The else if Statement

Use the else if statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

Example

If time is less than 10:00, create a "Good morning" greeting, if not, but time is less than 20:00, create a "Good day" greeting, otherwise a "Good evening":

```
if (time < 10) {  
    greeting = "Good morning";  
} else if (time < 20) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

The result of greeting will be:

Good day

The else if Example

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript if...else if...else Example</h2>
<p>Check the age category of a person:</p>
<p id="demo"></p>
<script>
let age = prompt("Enter your age:"); // User enters their age
age = Number(age); // Convert the input to a number
let message;
if (age < 13) {
  message = "You are a child.";
} else if (age >= 13 && age < 18) {
  message = "You are a teenager.";
} else if (age >= 18 && age < 60) {
  message = "You are an adult.";
} else {
  message = "You are a senior citizen.";
}
document.getElementById("demo").innerHTML = message;
</script>
</body>
</html>
```


The JavaScript Switch Statement

Use the switch statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, the default code block is executed.

Example: Switch

The `getDay()` method returns the weekday as a number between 0 and 6.
(Sunday=0, Monday=1, Tuesday=2 ..)

This example uses the weekday number to calculate the weekday name:

```
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";  
    break;  
  case 5:  
    day = "Friday";  
    break;  
  case 6:  
    day = "Saturday";  
}
```

The result of day will be:

Sunday

The break Keyword

When JavaScript reaches a **break** keyword, it breaks out of the switch block.

This will stop the execution inside the switch block.

It is not necessary to break the last case in a switch block. The block breaks (ends) there anyway.

Note: If you omit the break statement, the next case will be executed even if the evaluation does not match the case.

The default Keyword

The **default** keyword specifies the code to run if there is no case match:

Example: The `getDay()` method returns the weekday as a number between 0 and 6.

If today is neither Saturday (6) nor Sunday (0), write a default message:

```
switch (new Date().getDay()) {  
  case 6:  
    text = "Today is Saturday";  
    break;  
  case 0:  
    text = "Today is Sunday";  
    break;  
  default:  
    text = "Looking forward to the Weekend";  
}
```

The result of text will be:

Today is Sunday

JavaScript Loops

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Different Kinds of Loops

- **for** - loops through a block of code a number of times
- **while** - loops through a block of code while a specified condition is true
- **do/while** - also loops through a block of code while a specified condition is true

The For Loop

The for statement creates a loop with 3 optional expressions:

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

- Expression 1 is executed (one time) before the execution of the code block.
- Expression 2 defines the **condition** for executing the code block.
- Expression 3 is executed (every time) after the code block has been executed.

Example

```
for (let i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}
```

The For Loop Example

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript For Loop</h2>
<p id="demo"></p>
<script>
let text = "";
for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```

The While Loop

The while loop loops through a block of code as long as a specified condition is true.

Syntax

```
while (condition) {  
    // code block to be executed  
}
```

Example: In the following example, the code in the loop will run, over and over again, as long as a variable (i) is less than 10:

```
while (i < 10) {  
    text += "The number is " + i;  
    i++;  
}
```


The Do While Loop

The do while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {  
    // code block to be executed  
}  
while (condition);
```

Example: The example below uses a do while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

```
do {  
    text += "The number is " + i;  
    i++;  
}  
while (i < 10);
```

Functions

A JavaScript **function** is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

Example

```
// Function to compute the product of p1 and p2
function myFunction(p1, p2) {
  return p1 * p2;
}
```

Function Invocation

The code inside the function will execute when "something" invokes (calls) the function:

- When an **event** occurs (when a user clicks a button)
- When it is invoked (called) from **JavaScript** code
- Automatically (**self** invoked)

Function Return

- When JavaScript reaches a **return** statement, the function will **stop** executing.
- If the function was **invoked** from a statement, JavaScript will "**return**" to execute the code **after** the invoking statement.
- Functions often **compute** a return value. The return value is "**returned**" back to the "**caller**":

Example: Calculate the product of two numbers, and return the result:

```
// Function is called, the return value will end up in x
let x = myFunction(4, 3);
function myFunction(a, b) {
  // Function returns the product of a and b
  return a * b;
}
```

Reference Link: https://www.w3schools.com/js/js_functions.asp

Types of Functions

In JavaScript we have multiple types of functions:

- Regular Function
- Anonymous Function
- Arrow Function
- IIFE(Immediately Invoked Function Expression)

1. Regular Function

Definition

A **regular function** is a named function that can be reused and called multiple times.

A function with a **name**, defined using the function keyword.

Example:

```
function greet(name) {  
    return "Hello, " + name + "!";  
}  
console.log(greet("Alice"));
```

Example(Regular)

```
<body>
```

```
<h2>1. Regular Function</h2>
```

```
<p>A named function that can be called multiple times.</p>
```

```
<p id="regularOutput"></p>
```

```
<script>
```

```
  // Regular Function
```

```
  function greet(name) {
```

```
    return "Hello, " + name + "!";
```

```
  }
```

```
  document.getElementById("regularOutput").innerText =
```

```
  greet("Ammar");
```

```
</script>
```

```
</body>
```

2. Anonymous Function

Definition

An **anonymous function** is a function without a name, often assigned to a variable or passed as an argument.

A function **without a name**, usually stored in a variable

Example:

```
const sum = function(a, b) {  
    return a + b;  
};  
console.log(sum(5, 10));
```


Example(Anonymous)

```
<body>  
  <h1>Anonymous Function in JavaScript</h1>
```

<p>An **anonymous function** is a function without a name. It is often assigned to a variable and can be used as a function expression.</p>

```
<p id="anonymous-function"></p>
```

```
<script>  
  // Anonymous Function Example  
  const sum = function(a, b) {  
    return a + b;  
  };  
  
  // Display output  
  document.getElementById("anonymous-function").innerText = "Sum of  
7 and 8: " + sum(7, 8);  
</script>  
</body>
```

object in c++ vs JavaScript

Definition

In C++, **objects** follow a strict Object-Oriented Programming (OOP) model where everything must be planned in **advance** using classes. This ensures that every object has a fixed structure.

In contrast, JavaScript treats **objects** as flexible data containers that can be created and modified at **runtime**. This reflects a shift from **rigid** design (C++) to **adaptable** design (JavaScript), especially useful in web development where data structures change **frequently**.

Key Idea

- Object = combination of **data + behavior**
- Used to represent **real-world entities**

In C++

- Object = **instance of a class**
- Requires **blueprint (class)** before creation

In JavaScript

- Object = **collection of key-value pairs**
- Can be created **directly without class**

object creation mechanism

C++ Approach

- Class must be defined first
- Object is created from class

```
class Student {  
public:  
    string name;  
};
```

```
Student s1;
```

JavaScript Approach

- Object can be created instantly
- No class required (basic level)

```
let student = {  
    name: "Ammar"  
};
```

object creation mechanism

C++ enforces a **top-down design approach**, meaning developers must first **define** the structure (class) and then **create** objects. This reduces **ambiguity** and improves **maintainability**.

JavaScript, however, allows **bottom-up creation**, where objects are formed **directly** based on need. This makes **development faster** but requires **discipline** from developers to maintain consistency in large applications.

Typing and Structure

C++

- **Statically typed**
- Structure fixed at compile time
- Cannot add new properties later

`s1.age = 22;` // Not allowed if not defined

JavaScript

- **Dynamically typed**
- Structure can change anytime
- Properties can be added/removed

`student.age = 22;` // Allowed

Typing and Structure

C++ ensures safety by checking everything at **compile** time, which reduces **runtime** errors and enforces discipline.

JavaScript sacrifices this **strictness** for flexibility, allowing developers to modify objects on the fly. This is powerful in web applications (e.g., handling API data), but it can lead to **unexpected bugs** if the structure is not managed carefully.

this keyword

In JavaScript, the **this** keyword refers to an object.

The **this** keyword refers to different objects depending on how it is used:

- In an **object method**, **this** refers to the **object**.
- In a **function**, **this** refers to the **global object**.
- In an **event**, **this** refers to the element that received the event.

Note: **this** is not a variable. It is a **keyword**. You cannot change the value of **this**.

this in a Method

When used in an **object method**, **this** refers to the **object**.

In the example give on this page, **this** refers to the person **object**.

Because the **fullName method** is a **method** of the person object.

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  id      : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```


this in Event Handlers

In HTML event handlers, **this** refers to the HTML element that received the event:

Example

```
<button onclick="this.style.display='none'">
```

Click to Remove Me!

```
</button>
```

Example 1

Task: Create an object student with:

- name
- age
- method showInfo()

Inside method, print:

```
console.log(this.name);
```

Challenge:

- Create another object student2
- Assign student.showInfo to it
- Call student2.showInfo()

Example 2

Task: Create a button

```
<button onclick="changeColor()">Click Me</button>
```

```
function changeColor() {  
  this.style.backgroundColor = "red";  
}
```

Challenge:

- Print this in console
- Change text instead of color

Example 3

Fix the Bug:

```
<button onclick="handleClick()">Click</button>
```

```
<script>  
function handleClick() {  
  this.style.color = "blue";  
}  
</script>
```

Task

- Why does this NOT work properly?
- Fix it using:
 - event
 - OR inline this

3. Arrow Function

Definition

An **arrow function** provides a **concise** syntax for functions using `=>` and does not have its own **this** context.

Example:

```
const multiply = (x, y) => x * y;  
console.log(multiply(4, 3));
```

3. Arrow Function (Before/After)

Before Arrow:

```
hello = function() {  
  return "Hello World!";  
}
```

With Arrow Function:

```
hello = () => {  
  return "Hello World!";  
}
```

It gets **shorter**! If the function has only **one** statement, and the statement returns a value, you can remove the **brackets** and the **return** keyword:

Arrow Functions Return Value by Default:

```
hello = () => "Hello World!";
```

3. Arrow Function (Parameters)

Note: This works only if the function has **only one** statement.

If you have **parameters**, you pass them **inside** the parentheses:

Arrow Function With Parameters:

```
hello = (val) => "Hello " + val;
```

In fact, if you have only **one parameter**, you can **skip** the **parentheses** as well:

Arrow Function Without Parentheses:

```
hello = val => "Hello " + val;
```

Example (Arrow)

```
<body>  
  <h1>Arrow Function in JavaScript</h1>
```

```
  <p>An arrow function is a shorter syntax for writing functions  
  using `=>`. It does not have its own `this` context.</p>
```

```
  <p id="arrow-function"></p>
```

```
  <script>  
    // Arrow Function Example  
    const multiply = (x, y) => x * y;  
  
    // Display output  
    document.getElementById("arrow-function").innerText =  
    "Multiplication of 6 and 5: " + multiply(6, 5);  
  </script>  
</body>
```


Regular vs Anonymous vs Arrow

Differences:

- **Syntax:** Arrow functions are more **concise** compared to regular or anonymous functions.
- **this Binding:** Regular and anonymous functions have their **own this**, whereas arrow functions **inherit** this from their **surrounding** scope.
- **Usage:** Regular functions are used for named function declarations, anonymous functions for **callbacks**, and arrow functions for concise function expressions.

Task

Create a webpage with two buttons: “Show Name” and “Change Color”.

Use a **regular function inside an object** to display a name using this.

Use an **anonymous function** for a click event and log a message.

Use an **arrow function** to change the background color.

Observe how this behaves in each case.

Explain which function type works best with this and why.

Callbacks

A **callback** is a **function** that is passed as an **argument** to **another function** and is **executed** later, usually after a **certain** task is completed.

```
function greet(name, callback) {  
  console.log("Hello, " + name);  
  callback();  
}  
  
function sayBye() {  
  console.log("Goodbye!");  
}  
  
greet("Alice", sayBye);
```

// Output:

// Hello, Alice

// Goodbye!

Callbacks Example

```
<body>
  <h2>JavaScript Callback Function Example</h2>
  <p>Check the console for output.</p>

  <script>
    // Function that takes a name and a callback function
    function greet(name, callback) {
      console.log("Hello, " + name);
      callback(); // Execute the callback function
    }

    // Callback function
    function sayBye() {
      console.log("Goodbye!");
    }

    // Calling greet function with a callback
    greet("Alice", sayBye);
  </script>
</body>
```

JavaScript Events

- HTML events are "**things**" that happen to HTML elements.
- When JavaScript is used in HTML pages, JavaScript can "**react**" on these events.

HTML Events

- An HTML event can be something the browser **does**, or something a user does.

Here are some examples of HTML events:

- An HTML web page has finished **loading**
- An HTML **input field** was changed
- An HTML button was **clicked**

Single Quote or Double Quote

With single quotes:

```
<element event='some JavaScript'>
```

With double quotes:

```
<element event="some JavaScript">
```

- In the following example, an onclick attribute (with code), is added to a <button> element:

Example

```
<button onclick="document.getElementById('demo').innerHTML =  
Date()">The time is?</button>
```

Event Example

```
<!DOCTYPE html>  
<html>  
  <body>  
    <h1>JavaScript HTML Events</h1>  
    <h2>The onclick Attribute</h2>  
  
    <button onclick="this.innerHTML=Date()">The time is?</button>  
  
  </body>  
</html>
```

Common HTML Event

Here is a list of some common HTML events:

Events:

onchange: An HTML element has been changed

onclick: The user clicks an HTML element

onmouseover: The user moves the mouse over an HTML element

onmouseout: The user moves the mouse away from an HTML element

onkeydown: The user pushes a keyboard key

onload: The browser has finished loading the page

Example: EVENT folder

Reference Link: [W3Schools JavaScript Reference HTML DOM Events](#).

ES6+ Features

Reference Link: https://www.w3schools.com/js/js_es6.asp

Error Handling

Throw, and Try...Catch...Finally

- The **try** statement defines a code block to run (to try).
- The **catch** statement defines a code block to handle any error.
- The **finally** statement defines a code block to run regardless of the result.
- The **throw** statement defines a custom error.

JavaScript try and catch

The **try** statement **allows** you to define a block of code to be tested for errors while it is being executed.

The **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.

The JavaScript statements try and catch come in pairs:

```
try {  
    Block of code to try  
}  
catch(err) {  
    Block of code to handle errors  
}
```

Example Try and Catch

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Error Handling</h2>
<p>How to use <b>catch</b> to display an error.</p>
<p id="demo"></p>
<script>
try {
  adddlert("Welcome guest!");
}
catch(err) {
  document.getElementById("demo").innerHTML = err.message;
}
</script>
</body>
</html>
```

The throw Statement

- The throw statement allows you to create a **custom error**.
- Technically you can **throw an exception (throw an error)**.

The exception can be a JavaScript String, a Number, a Boolean or an Object:

```
throw "Too big";    // throw a text  
throw 500;          // throw a number
```

Note: If you use **throw** together with **try** and **catch**, you can **control program flow** and generate custom error messages.

Example(Throw)

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript try catch</h2>
<p>Please input a number between 5 and 10:</p>
<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>
<p id="p01"></p>
<script>
function myFunction() {
  const message = document.getElementById("p01");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;
  try {
    if(x.trim() == "") throw "empty";
    if(isNaN(x)) throw "not a number";
    x = Number(x);
    if(x < 5) throw "too low";
    if(x > 10) throw "too high";
  }
  catch(err) {
    message.innerHTML = "Input is " + err;
  }
}
</script>
</body>
</html>
```

The finally Statement

The **finally** statement lets you execute code, after try and catch, regardless of the result:

Syntax

```
try {  
    Block of code to try  
}  
catch(err) {  
    Block of code to handle errors  
}  
finally {  
    Block of code to be executed regardless of the try / catch result  
}
```

Example (finally)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript finally Example</title>
</head>
<body>
  <h1>JavaScript try...catch...finally Example</h1>
  <button onclick="testErrorHandling()">Click to Test Error</button>
  <p id="error-message"></p>
  <script>
    function testErrorHandling() {
      try {
        // Simulating an error
        let result = 10 / unknownVariable; // `unknownVariable` is not defined
      } catch (error) {
        // Handling the error
        document.getElementById("error-message").innerText = "Error: " + error.message;
      } finally {
        // Always executes
        console.log("Finally block executed!");
        alert("Finally block executed!");
      }
    }
  </script>
</body>
</html>
```


Task

Create a webpage with a button “**Check Age**” and an input field.

Use **try...catch** to handle errors when the user enters invalid input.

Use **throw** to display custom errors (e.g., “Age must be a number” or “Age must be ≥ 18 ”).

Display error messages on the screen instead of crashing.

Test with empty, text, and valid inputs.

Explain when throw is used instead of default errors.